



Department of Information Science & Engineering

Continuous Internal Evaluation Test II - ODD Semester 2025 - 26

Scheme and Solutions of Evaluation

Course Title: **Software Testing**

Course Code: **IS722T2I**

Prepared By: **Mrs. Shwetha S Shetty / Mrs. Aishwarya M Bhat**

Semester: **VII**

Q. No.		Solution	Marks Distribution
1.	a.	Slice based testing: A program slice is a set of program statements that contributes to or affects a value for a variable at some point in the program. Example- Commission problem 1. Program Commission (INPUT, OUTPUT) 2. Dim locks, stocks, barrels As Integer 3. Dim lockPrice, stockPrice, barrelPrice As Real 4. Dim totalLocks, totalStocks, totalBarrels As Integer 5. Dim lockSales, stockSales, barrelSales As Real 6. Dim sales, commission As Real 7. lockPrice = 45.0 8. stockPrice = 30.0 9. barrelPrice = 25.0 10. totalLocks = 0 11. total Stocks = 0 12. totalBarrels = 0 13. Input (locks) 14. While NOT(locks = -1) "Loop' condition uses -1 to indicate end of data 15. Input (stocks, barrels) 16. total Locks = totalLocks + locks 17. totalStocks = totalStocks + stocks 18. totalBarrels = totalBarrels + barrels 19. Input (locks) 20. EndWhile	1.0 3.0

	21. Output("Locks sold: " totalLocks) 22. Output("Stocks sold: ", totalStocks) Output("Barrels sold: ", totalBarrels) 23. lockSales = lockPrice * totalLocks 24. stockSales = stockPrice * totalStocks 25. barrelSales = barrel Price * totalBarrels 26. sales = lockSales + stockSales + barrelSales 27. Output("Total sales: ", sales) 28. If (sales> 1800.0) 29. Then 30. commission = 0.10 * 1000.0 31. commission = commission + 0.15 * 800.0 32. commission = commission + 0.20 * (sales-1800.0) 33. Else If (sales> 1000.0) 34. Then 35. commission = 0.10 * 1000.0 36. commission = commission + 0.15 * (sales-1000.0) 37. Else 38. commission = 0.10 * sales 39. EndIf 40. EndIf 41. Output("Commission is \$", commission) 42. End Commission 1. Slices on the locks: Slices on the locks variable show why it is potentially fault-prone. It has a P-use at node 14 and a C-use at node 16 and has two definitions, the I-defs at nodes 13 and 19. S1: S(locks, 13) = {13} S2: S(locks, 14) = {13, 14, 19, 20} S3: S(locks, 16) = {13, 14, 19, 20} S4: S(locks, 19) = {19} 2. Slices for stocks and barrels: S5: S(stocks, 15) = {13, 14, 15, 19, 20} S6: S(stocks, 17) = {13, 14, 15, 19, 20} S7: S(barrels, 15) = {13, 14, 15, 19, 20} S8: S(barrels, 18) = {13, 14, 15, 19, 20}	2.0 2.0 2.0
--	--	----------------------------------

3. Slices for totalLocks:

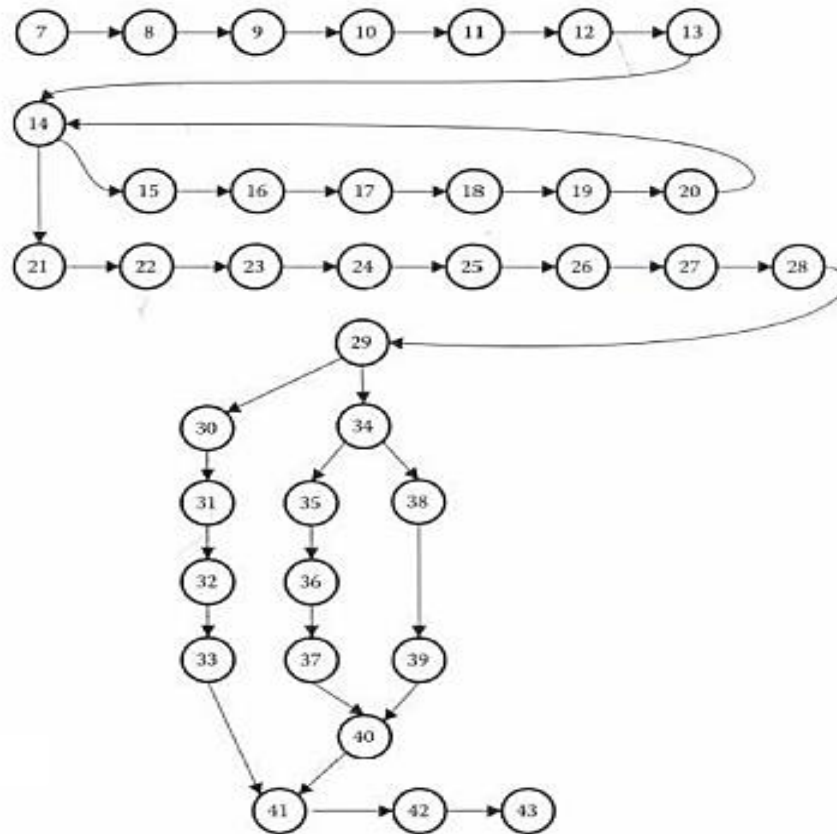
S9: $S(\text{totalLocks}, 10) = \{10\}$

S10: $S(\text{totalLocks}, 16) = \{10, 13, 14, 16, 19, 20\}$

S11: $S(\text{totalLocks}, 21) = \{10, 13, 14, 16, 19, 20\}$

- b. **du path**-A definition-use path with respect to a variable v (du-path) is a path in $\text{PATHS}(P)$ such that for some $v \in V$, there are define and usage nodes $\text{DEF}(v, n)$ and $\text{USE}(v, n)$ such that m and n are the initial and final nodes of the path.

Example- Commission problem

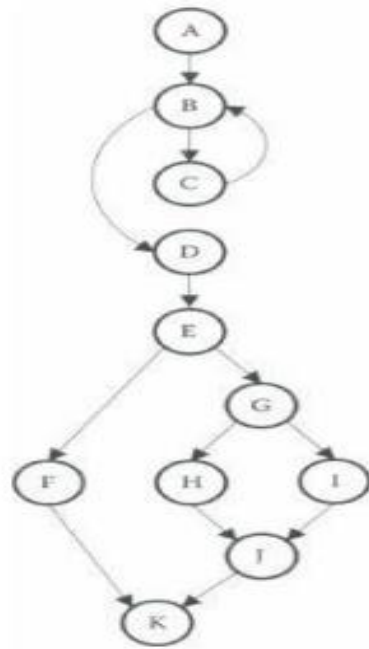


Program graph of the commission program

1.0

2.0

2.0



DD-path graph of commission program

du paths for Stocks-

P1 = <15,17>

du path for Locks-

P2 = <13, 14>

P3 = <13, 14, 15, 16>

P4 = <19, 20, 14>

P5 = <19, 20, 14, 15, 16>

du paths for total Locks-

P6 = <10,11,12,13,14,15,16>

P7 = <10,11,12,13,14,15,16,17,18,19,20,14,21>

P8 = <10,11,12,13,14,15,16,17,18,19,20,14,21,22,23,24>

P9 = <16,17,18,19,20,14,21>

P10 = <16,17,18,19,20,14,21,22,23,24>

1.0

2.0

2.0

2.

a.

i) Scaffolding- Code developed to facilitate testing is called scaffolding. Scaffolding refers to the creation of temporary code or support structures that help test individual units or modules of a program.

Scaffolding may include test drivers, test harnesses and stubs in addition to program instrumentation and support for recording and managing test execution.

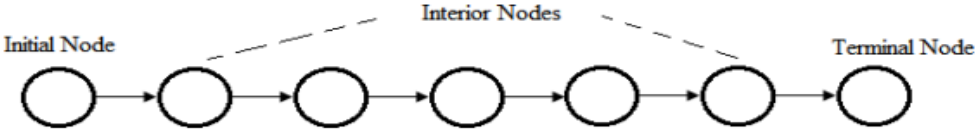
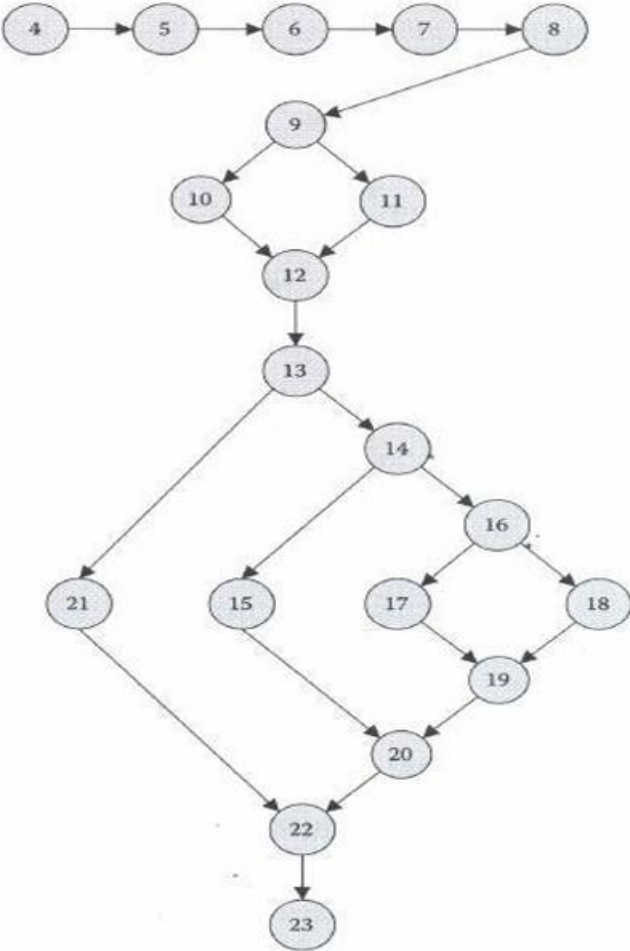
ii) Test Oracles-Test oracle is a Software that applies a pass/fail criterion to a program execution. Used to determine whether a software test has passed or failed by comparing the actual output with the expected output.

iii) Self checks- An oracle can also be written as *self-checks*. The program checks itself and acts as its own oracle. Instead of relying on an external oracle (like predefined expected outputs), the

2.0

2.0

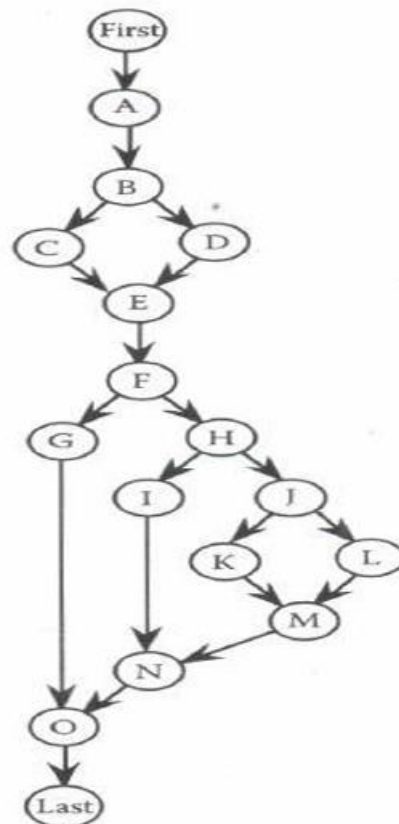
2.0

	program has built-in checks — like assertions, error-handling— to determine if something has gone wrong. iv)Capture-You record the interactions, inputs, or events that happen during a real use of the software (e.g., user actions, network messages, system calls). Sometimes there is no alternative to human input and observation. Even if we separate testing program functionality from GUI, some testing of the GUI is required. We can at least cut repetition of human testing. v)Replay-You automatically replay those recorded actions later to test if the system behaves the same way or detect bugs. Capture a manually run test case, replay it automatically.	2.0 2.0
b.	DD-Paths in terms of paths of nodes in a directed graph (Chains)- A chain is a path in which the initial and terminal nodes are distinct, and every interior node has indegree = 1 and outdegree = 1  DD-Path Graph for Triangle problem- Triangle problem program- Program graph-  Program graph of Triangle problem	1.0 2.0 2.0

DD paths-

Program Graph Nodes	DD path Name	Case of Definition
4	First	1
5-8	A	5
9	B	3
10	C	4
11	D	4
12	E	3
13	F	3
14	H	3
15	I	4
16	J	3
17	K	4
18	L	4
19	M	3
20	N	3
21	G	4
22	O	3
23	Last	2

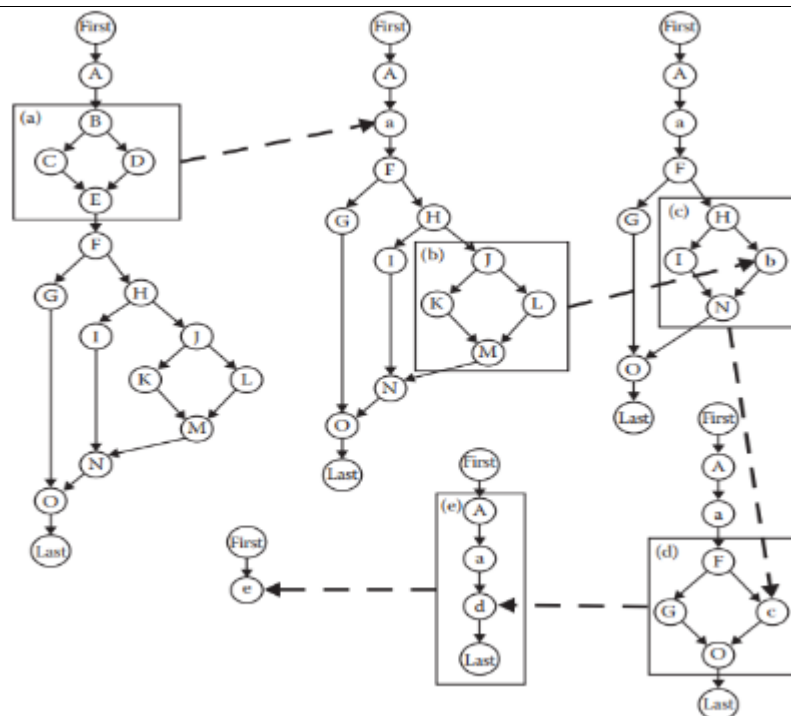
1.0



2.0

DD-Path graph for the Triangle Program

2.0



Condensing with respect to the structured programming constructs

3.	a.	Software testing is a crucial phase in the software development lifecycle that ensures a product is reliable, functional, secure, and ready for release. Below are the key reasons why software testing is important: 1. Ensures Software Quality- Confirms that the software performs as expected under various conditions. Verifies that all requirements (functional and non-functional) are met. 1.0 2. Detects and Fixes Bugs Early- Early detection of defects reduces the cost and effort of fixing them later. Helps maintain the stability and integrity of the product throughout development. 1.0 3. Improves Security- Identifies vulnerabilities that could be exploited by attackers. 1.0 4. Enhances User Satisfaction- A well-tested product is more likely to be user-friendly, efficient, and free of errors. Leads to higher user confidence and better customer retention. 1.0 5. Supports Cost Efficiency- Prevents costly post-release bugs and system failures. Saves money by minimizing the risk of project rework, recalls, or legal issues. 1.0 Relation of Test automation to Software development aspects- 1. Architecture- Well-structured and modular architectures allow easier testing of individual components, while poor design can make creating and maintaining automated tests difficult. 1.0 2. Working Practices- Test automation fits into modern development practices like Agile and DevOps. It supports continuous integration (CI) and continuous delivery (CD), providing rapid feedback to developers and helping to catch bugs early. 1.0 3. Quality and Productivity- Automated testing ensures consistent software quality by detecting defects quickly and reliably. It also saves time and effort by reducing repetitive manual testing, allowing developers and-testers to focus on more critical tasks. 1.0 4. Business Structure- Teams need to view automation as a shared responsibility rather than leaving it to testers alone. This mindset ensures smoother implementation and better results. 1.0 5. Documentation- Good documentation is crucial for test automation. It helps teams understand how the test framework works and how to use or update it. 1.0
----	----	---

4.	a.	Record and Playback- • The first attempt at test automation aimed to replace manual testing is by recording and replaying user actions. • One way to achieve is to record the actions of the manual tester and then play them over and over again. • This is commonly done through UI interaction recordings, or alternatively by capturing network traffic (e.g., HTTP requests). • How Record and Playback Works- Tester performs actions; the tool records them. Recorded actions are saved and can be replayed automatically. Intended to reduce repetitive manual regression testing. • While it's simple to record the actions that the manual tester does and replay them, it's much trickier to detect bugs in an automated manner. • Screenshot comparison- One approach to automatically detect bugs is to compare the image on the screen to the expected image that was recorded. • Drawback- There could be differences in screen resolution, differences in date and time that appear on the screen, differences in any data that appear on the screen but is not relevant to the test, etc. • HTTP network traffic comparison- Drawback-some of the data may have legitimate differences. • Even if those technical drawbacks can be addressed by tools, there's still one big drawback- every legitimate change in the application will be considered as a failure, making it hard to distinguish between false positives and actual failures. • Human vs Machine in Test Evaluation- -- Humans use judgment and context to interpret changes. --Machines treat all differences equally—whether they are bugs or valid changes	2.0 2.0 2.0 2.0 2.0
----	----	---	--